Beyond Syscalls

Inside the World of eBPF

Linux Festival · Amirkabir University of Technology

Mohammad Heidari

About me

Mohammad Heidari
Started as a **Go developer**, now a **Platform Engineer**
Multi-tenant production Kubernetes in private clouds: telecom, fintech, health tech

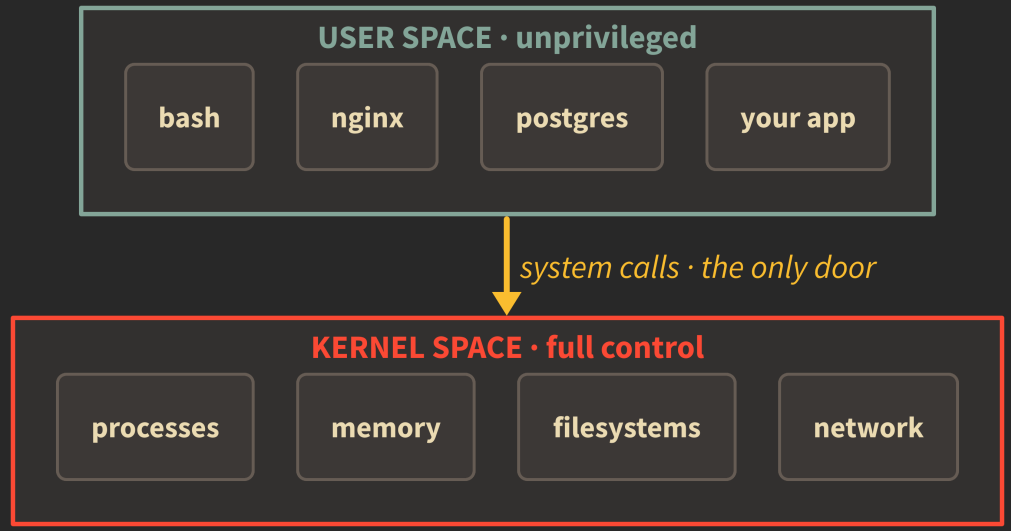
Not a kernel developer. That's the point: eBPF is no longer only for kernel developers.

Agenda

1. **Why eBPF exists**
2. **The story of BPF**
3. **Beyond syscalls**
4. **How eBPF works**
5. **Hook points**
6. **Networking**
7. **eBPF in production**
8. **Getting started**

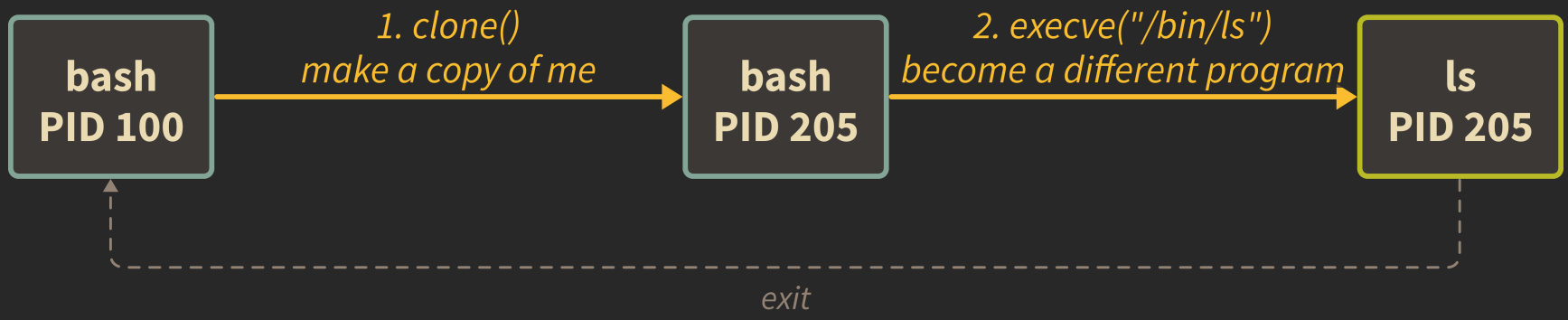
1 · Why eBPF Exists

Two worlds



Programs **cannot** touch hardware, disks, the network, or other processes. They **ask** the kernel, through a **system call**.

How every program starts: fork + execve



Remember this: at execve entry, the child is **still called bash**. We'll see it live soon.

Watching through the window

To **see** what the kernel is doing, we had four families of tools:

Tool	How it sees	The catch
/proc, /sys	reads counters the kernel exports	only what's exported, only when you look
strace	stops the process at every syscall	painfully slow
tcpdump	copies packets to user space	lots of copying
perf, ftrace, SystemTap, DTrace	kernel-developer tools	hard, risky, or not on Linux

strace: stop the world at every syscall

```
echo "without strace:"
time dd of=null bs=1 count=200K status=none
echo "with strace:"
time strace -o null dd of=null bs=1 count=200K status=none
```

──────────────────────── [finished] ────────────────────────

```
without strace:
0.152 seconds
with strace:
6.750 seconds
```

The kernel **stops** the process at every syscall entry **and** exit, wakes up strace, waits, resumes.

Want to change the kitchen?

To **change** how the kernel behaves, you had two options:

- **Patch the kernel upstream**: ask the head chef to change the recipe → it takes **years** to reach your servers
- **Write a kernel module**: sneak your own cook into the kitchen, unsupervised → one bug **crashes the whole machine**, and it breaks on every kernel upgrade

Networking the old way

▒▒▒ iptables / netfilter

- Rules in chains, checked **one by one**
- Kubernetes kube-proxy writes rules **per service**
- Thousands of services → thousands of rules on every node

▒▒▒ Kernel bypass (DPDK)

- Take the network card **away from the kernel**
- Very fast, but you **lose** the kernel's TCP/IP stack and tools
- Burns CPU cores just polling the card

Every option has a ✘

	Safe	Fast	Runtime change	Sees all	Upgrades
/proc polling	✔	✔	✔	✘	✔
strace	✔	✘	✔	✘	✔
tcpdump	✔	✘	✔	✘	✔
Kernel patch	✔	✔	✘	✔	✔
Kernel module / SystemTap	✘	✔	✔	✔	✘
DPDK bypass	✔	✔	✘	✘	✔
Sidecars / LD_PRELOAD	✔	✘	✔	✘	✔

We wanted all five ✔ at the same time.

When kernel code goes wrong

July 19, 2024: a faulty update to a security product's **kernel driver** crashed about **8.5 million** Windows machines.
Airlines, hospitals, and banks went down.

Kernel code is powerful, and one mistake takes down everything.
What if the kernel could *check* your code before running it?

The idea: a programmable kernel

▒▒▒ Your browser

- Runs untrusted **JavaScript** from any website
- **Sandboxed**, **checked**, **JIT-compiled**
- Changed what a web page can be

▒▒▒ Linux with eBPF

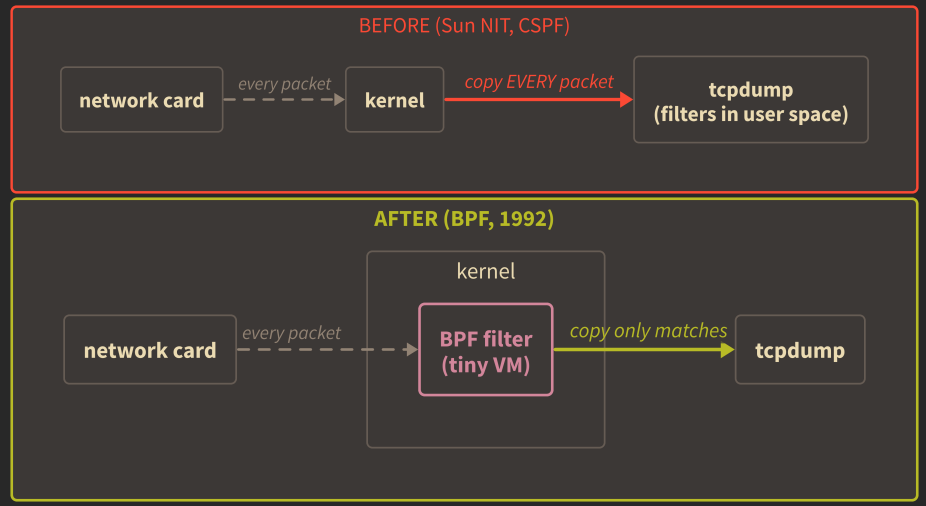
- Runs your small programs **inside the kernel**
- **Verified** before loading, **JIT-compiled** to native code
- Attached to events, with **no reboot**

> "eBPF does to Linux what JavaScript does to HTML. (Sort of.)"
> — Brendan Gregg, 2019

2 · The Story of BPF

1992: watching packets was too expensive

Van Jacobson's team at Lawrence Berkeley Lab built **tcpdump** (1988) to debug TCP on the early Internet. But the OS copied **every** packet to it: too slow, and packets were lost.



Their fix: Steven McCanne & Van Jacobson, *The BSD Packet Filter* (USENIX, 1993): run the filter **inside the kernel**. The filter is a program for a **made-up CPU**, a "virtual machine" like Java's JVM (not like KVM): a few simple instructions, executed by a small loop of kernel code. Result: up to **100 times faster** than copying every packet to user space (Sun's NIT).

BPF is still inside your tcpdump

```
tcpdump -d 'icmp'
```

```
──────────────────── [finished] ────────────────────

(000) ldh      [12]
(001) jeq      #0x800           jt 2    jf 5
(002) ldb      [23]
(003) jeq      #0x1             jt 4    jf 5
(004) ret      #262144
(005) ret      #0
```

How to read it (syntax from the kernel docs, networking/filter.rst): [12] = byte 12 of the packet · # = a number · 0x = hexadecimal

Virtual CPU instruction	Meaning
ldh [12]	EtherType
jeq #0x800	IPv4?
ldb [23]	protocol
jeq #0x1	ICMP?
ret #262144	**keep**
ret #0	**drop**

BPF shipped inside the **BSD** kernel. **Linux** adopted the same instruction set in **1997** (Linux Socket Filter), so the filters tcpdump compiles run unchanged on both.

2014: eBPF, "extended" BPF

Why: at PLUMgrid, a software-defined networking startup, Alexei Starovoitov wanted to load **networking programs into the kernel**, safely, without kernel modules. He rebuilt BPF as a general engine:

	Classic BPF (1992)	eBPF (2014)
Registers	2 × 32-bit	**11 × 64-bit**
Loops	none	bounded loops (2019)
State between runs	none	**maps**
Calls into the kernel	none	**helper functions**
Safety	simple rules	the **verifier**
Runs	packet filters	**almost any kernel event**

> "You cannot trust the compiler… When something gets loaded in the kernel,
> it shouldn't be trusting the user space. So that's how the verifier came about."
> — Alexei Starovoitov, *eBPF: Unlocking the Kernel* (2023)

How eBPF got into Linux

- **2013:** first patch: a brand-new instruction set plus an LLVM backend, a "huge patch bomb". **Nobody looked.**
- Advice from Red Hat: "*make it similar to what already exists*" → reshaped as **extended BPF**
- With **Daniel Borkmann** (Red Hat): step one only **replaces the old tcpdump interpreter**, so existing filters run **faster**
- Networking maintainer David Miller: "*Applied, thanks.*" (Linux 3.15, 2014)
- Networking still pushed back ("*Open vSwitch already does this*") → a second user: **tracing**, with Netflix
- **Linux 3.18 (December 2014):** the `bpf()` syscall. Then **"Lego brick by Lego brick"**

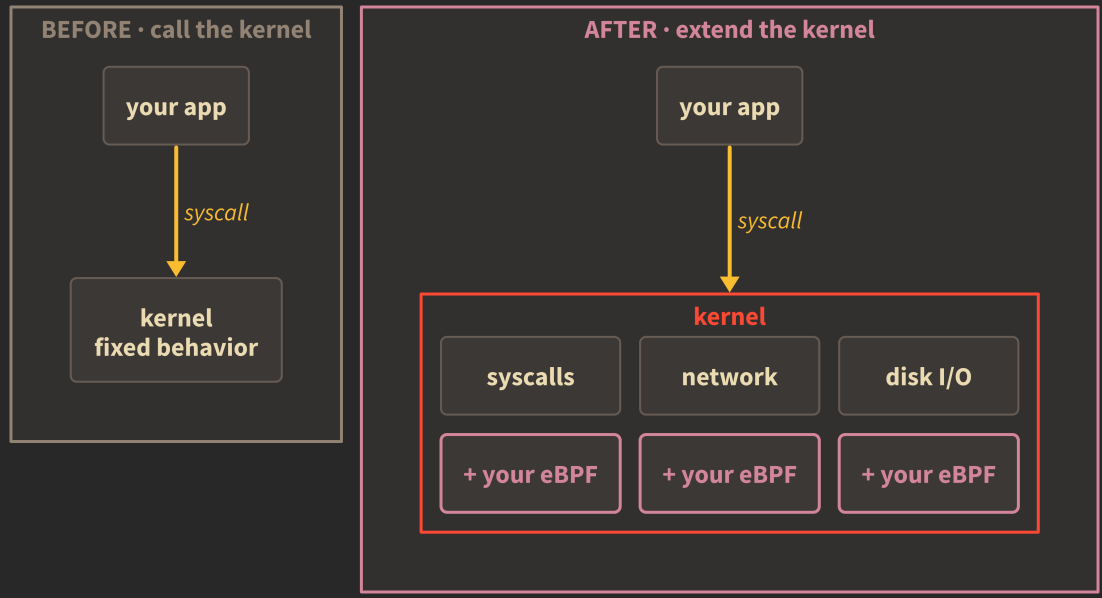
A revolutionary idea, merged as **small, useful, boring steps**. Today "BPF" means eBPF; the old one is "cBPF".

Then it exploded

1992 BPF paper (tcpdump)	1997 Linux Socket Filter	2011 BPF JIT	2012 seccomp-bpf	2014 eBPF bpf() in 3.18
2015 kprobes · BCC	2016 XDP	2018 BTF · bpftrace Katran	2020 BPF LSM ring buffer	2021+ eBPF Foundation Cilium graduates

3 · Beyond Syscalls

From calling the kernel to extending it



You don't just *call* the kernel anymore. You *extend* it, safely, at runtime.

So… do you know what is running on your laptop right now?

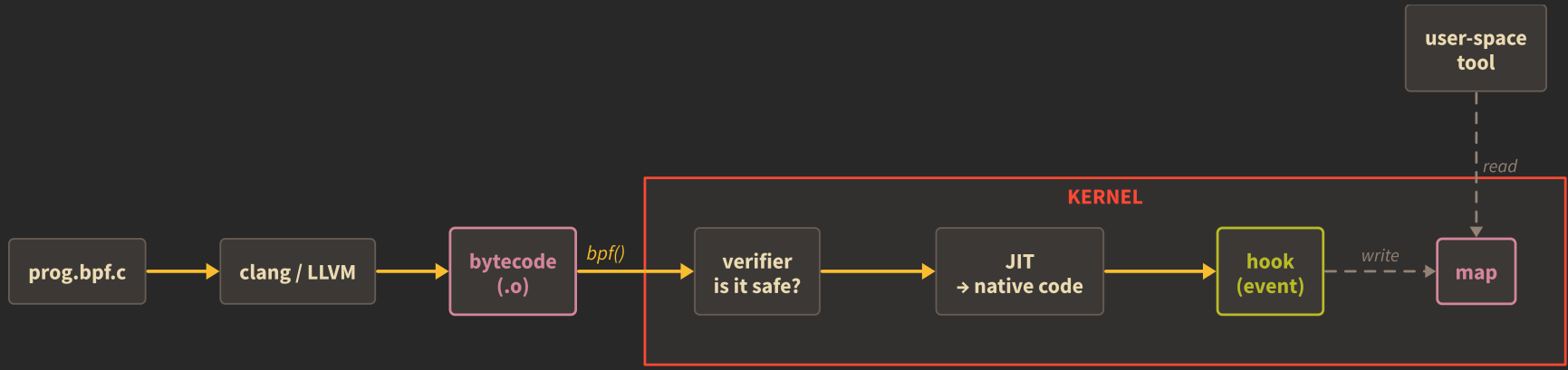
Live: ask the kernel

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_execve {
  printf("%s -> %s\n", comm, str(args->filename));
}'
```

One line. **Every program started on this machine**, by any user, shows up.

4 · How eBPF Works

The big picture



Write → **compile** → **load** → **verify** → **JIT** → **attach** → runs on every event → shares data through **maps**

Step 1: bytecode for a virtual CPU

D1's tracer written in C (left), compiled by `clang -target bpf` into **bytecode** (right, `llvm-objdump -d`):

```c
SEC("tracepoint/syscalls/sys_enter_execve")
int trace_execve(struct trace_event_raw_sys_enter *ctx)
{
  char comm[16], file[64];
  bpf_get_current_comm(&comm, 16);
  bpf_probe_read_user_str(&file, 64,
                          (void *)ctx->args[0]);
  bpf_printk("%s -> %s", comm, file);
  return 0;
}
```

```
 0: r6 = r1
 1: r7 = r10
    …                ; stack setup
 5: call 0x10        ; helper 16
 6: r3 = *(u64 *)(r6 + 0x10)
    …
11: call 0x72        ; helper 114
    …
17: call 0x6         ; helper 6
18: w0 = 0x0
19: exit
```

A **virtual CPU**: 11 registers, a 512-byte stack · `call N` = call helper N (numbers in **hex**: `0x72` = 114) · **20 instructions**

Step 2: the verifier, the head chef inspects the recipe

Before your program runs **even once**, the kernel simulates **every possible path** through it:

- Does **every path end**? (no infinite loops)
- Is **every memory access in bounds**? (stack, maps, packets)
- Is every pointer **checked for NULL** before use?
- Does it only call **helpers allowed** for this program type?
- Does it read **only initialized** memory?
- Is it within the **size and complexity limits**?

If the verifier can't prove it's safe, the program is rejected. No exceptions.

The verifier says no

```c
SEC("tracepoint/syscalls/sys_enter_execve")
int stuck(void *ctx)
{
    while (1) { }      /* BUG: never ends */
    return 0;
}
```

```
$ sudo bpftool prog load verifier-no.bpf.o /sys/fs/bpf/stuck
0: (05) goto pc-1
...
infinite loop detected at insn 0
cur state: R1=ctx() R10=fp0
old state: R1=ctx() R10=fp0
```

Loops are fine if the verifier can **prove** they end: `for (int i = 0; i < 10; i++)`. A program in the kernel must **always finish**.

Step 3: JIT, cooked at kitchen speed

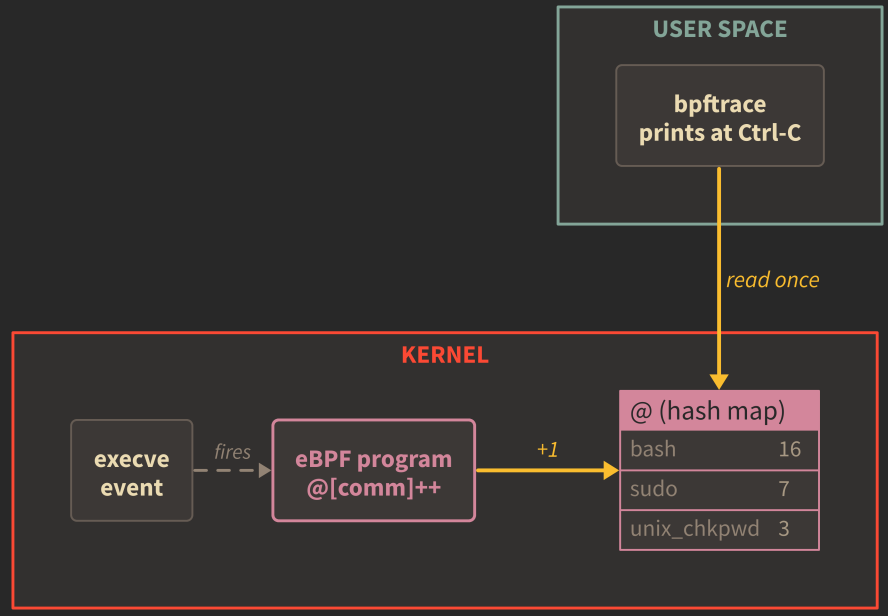
	Interpreter	JIT compiler
How	decodes each instruction, every time it runs	translates the whole program **once**
Speed	slow	**native machine code**

From my laptop (bpftool prog list):

```
xlated 768B  jited 448B
```

96 verified eBPF instructions → 448 bytes of real **x86-64** code. The same bytecode becomes **ARM64** code on a Raspberry Pi or a cloud ARM server.

Maps: sharing data with user space



Aggregate inside the kernel, copy out only the result. Types: hash · array · per-CPU · LRU · ring buffer · LPM trie (IP prefixes) · …

Helpers and kfuncs: the kernel's API for eBPF

An eBPF program **can't call any kernel function it likes**. It gets an approved list:

Helper	Does
bpf_get_current_comm	name of the current process
bpf_probe_read_user_str	safely copy a string from user memory
bpf_map_lookup_elem	read from a map
bpf_ktime_get_ns	current time
bpf_ringbuf_output	send an event to user space
bpf_redirect	send a packet to another interface

Ask the kernel itself (its type info, BTF; no root needed):

```
bpftool btf dump file /sys/kernel/btf/vmlinux | grep "'BPF_FUNC_" | grep -vc unspec
bpftool btf dump file /sys/kernel/btf/vmlinux | grep -c "DECL_TAG 'bpf_kfunc'"
```

[finished]

```
211
250
```

My kernel (6.18): **211 helpers** and **250 kfuncs**. Each program type may use only its own subset.

5 · Hook Points

Where can eBPF attach?

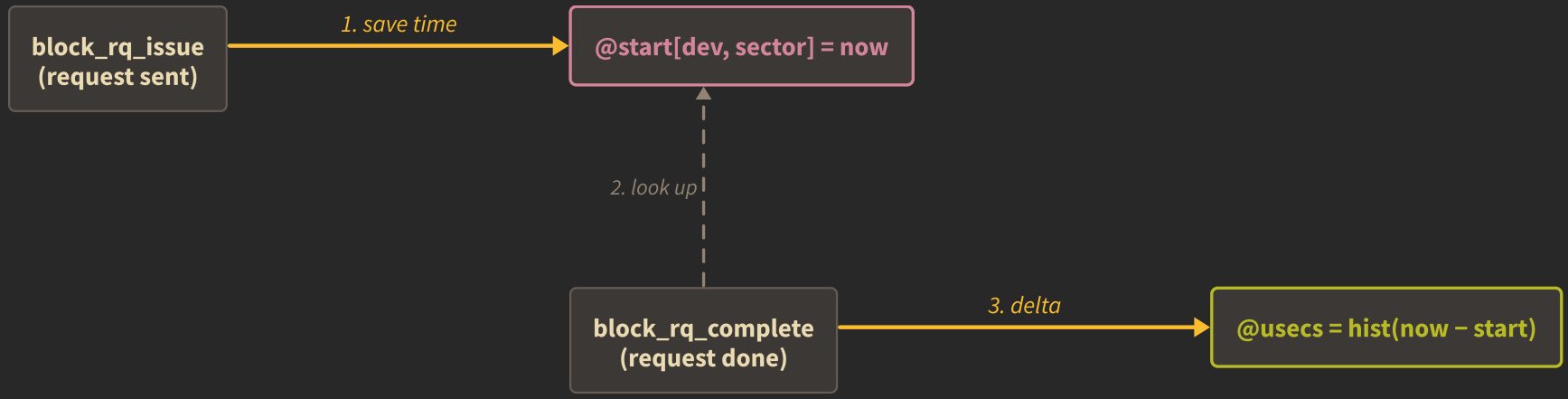
Layer	Hook	What it is
Applications	**uprobe** (user probe)	any function in a program or library
	USDT (user statically-defined tracing)	trace points an app's developers built in
System calls	**tracepoint**	official hooks placed by kernel developers
	seccomp (secure computing)	allow or block a process's syscalls
Kernel functions	**kprobe** (kernel probe)	(almost) any kernel function
	fentry / fexit	same, faster: on function entry / exit
Security	**LSM** (Linux Security Modules)	the kernel's "is this allowed?" checks
Network stack	**socket filter**	the data of one socket (tcpdump's filter)
	TC (Traffic Control)	packets in both directions, after the driver
	cgroup_skb	packets of one service or container
Network driver	**XDP** (eXpress Data Path)	packets inside the NIC driver, earliest point

Three ways into the kernel

	Tracepoint	kprobe	fentry / fexit
What	hook **placed on purpose** by kernel developers	**any** kernel function, patched at runtime	any function, via a BPF trampoline
Stable across kernels?	✔ yes	✘ no	✘ no
Overhead	low	higher	lowest
Since	Linux 4.7	Linux 4.1	Linux 5.5

Kitchen: tracepoints are the **official doors**. kprobes are **any window** you can open.

Measuring latency



Two events, one map, and a **histogram computed inside the kernel**.

Live: is the disk slow?

```
sudo bpftrace -e '
tracepoint:block:block_rq_issue { @start[args->dev, args->sector] = nsecs; }
tracepoint:block:block_rq_complete /@start[args->dev, args->sector]/ {
  @usecs = hist((nsecs - @start[args->dev, args->sector]) / 1000);
  delete(@start[args->dev, args->sector]);
}'
```

uprobes: every command, every user

kprobes hook the **kernel**. **uprobes** hook any function in any **binary or library**: the kernel patches it in memory, and the program never knows.

```
LIB=$(ldd $(readlink -f $(which bash)) \
      | awk '/libreadline/ {print $3}')
sudo bpftrace -e 'uretprobe:'"$LIB"':readline {
  printf("pid %-7d typed: %s\n", pid, str(retval));
}'
```

- uretprobe fires when readline() **returns**: its return value is the line just typed
- Every bash shell on the machine, **including root's**

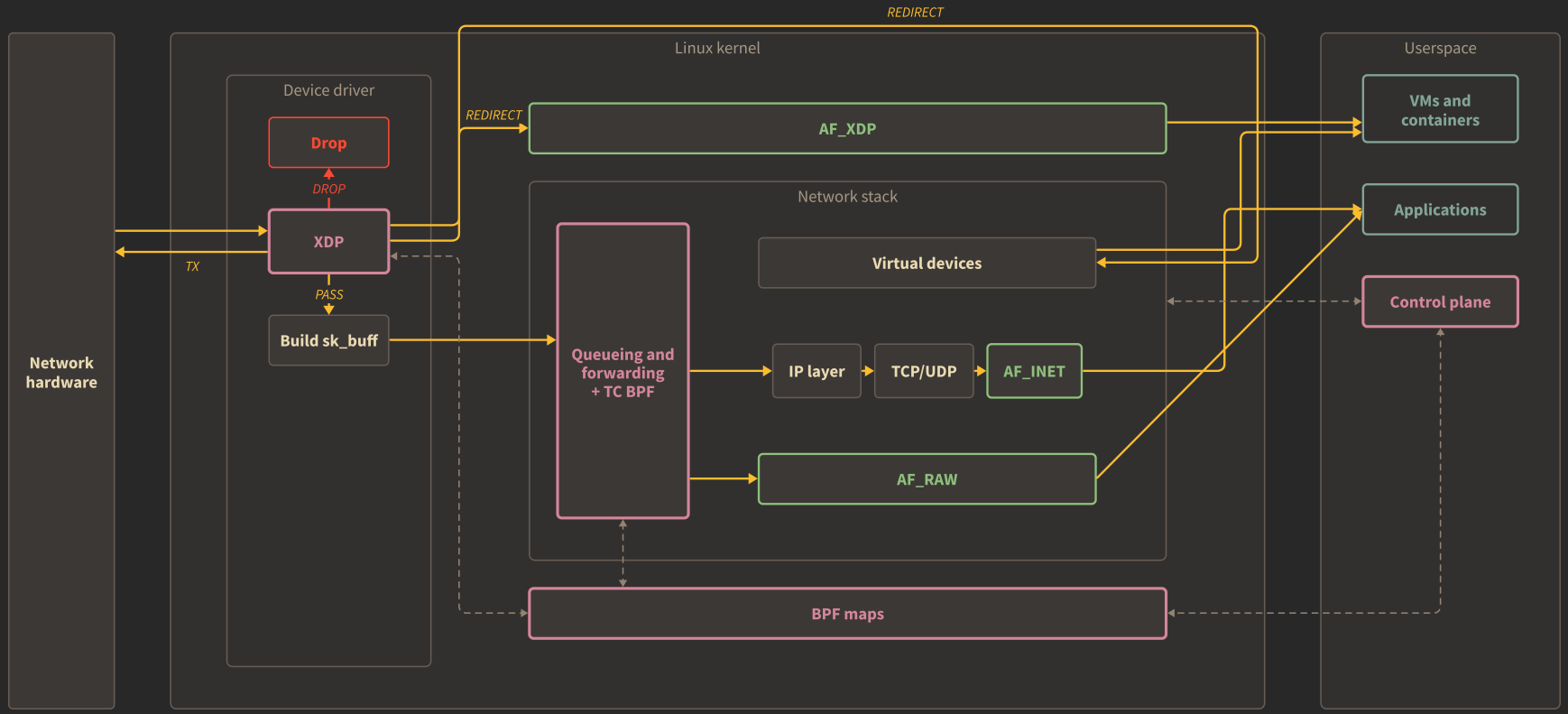
Why is it so cheap?

	strace (section 1)	eBPF
Stops the process?	at every syscall	**never**
Where the logic runs	user space, after a context switch	**inside the kernel**, native code
What crosses to user space	every event	only **results** (maps)

This is why eBPF can run in production, all the time.

6 · Networking

A packet's journey



On packet arrival, **before touching the packet data**, the driver runs an eBPF program in the **XDP** hook. It can **drop** the packet, send it back out the same interface (**TX**), **redirect** it (another interface, a VM, or user space via **AF_XDP**), or **pass** it to the normal stack, where a **TC BPF** hook can process it further. eBPF programs and user space share data through **BPF maps**. (Receive path only.)

Based on Figure 1, "The eXpress Data Path" (Høiland-Jørgensen et al., CoNEXT 2018)

XDP: decide at the door

Your program returns **one verdict** for every packet:

Verdict	What happens	Used for
XDP_DROP	dropped immediately	DDoS protection
XDP_PASS	continues to the normal stack	everything else
XDP_TX	sent back out the same card	load balancers
XDP_REDIRECT	sent to another card, CPU, or socket	fast forwarding, AF_XDP
XDP_ABORTED	dropped, with an error trace	bugs

Meta's **Katran** load balancer: every packet to facebook.com goes through XDP. **15 million packets/s**, about **50 ns** per packet, 10× faster than IPVS.

An XDP program: drop all ICMP

```c
SEC("xdp")
int drop_icmp(struct xdp_md *ctx)
{
    void *data     = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;

    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)
        return XDP_PASS;
    if (eth->h_proto != bpf_htons(ETH_P_IP))
        return XDP_PASS;

    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return XDP_PASS;

    return ip->protocol == IPPROTO_ICMP ? XDP_DROP : XDP_PASS;
}
```

Live: dropping packets

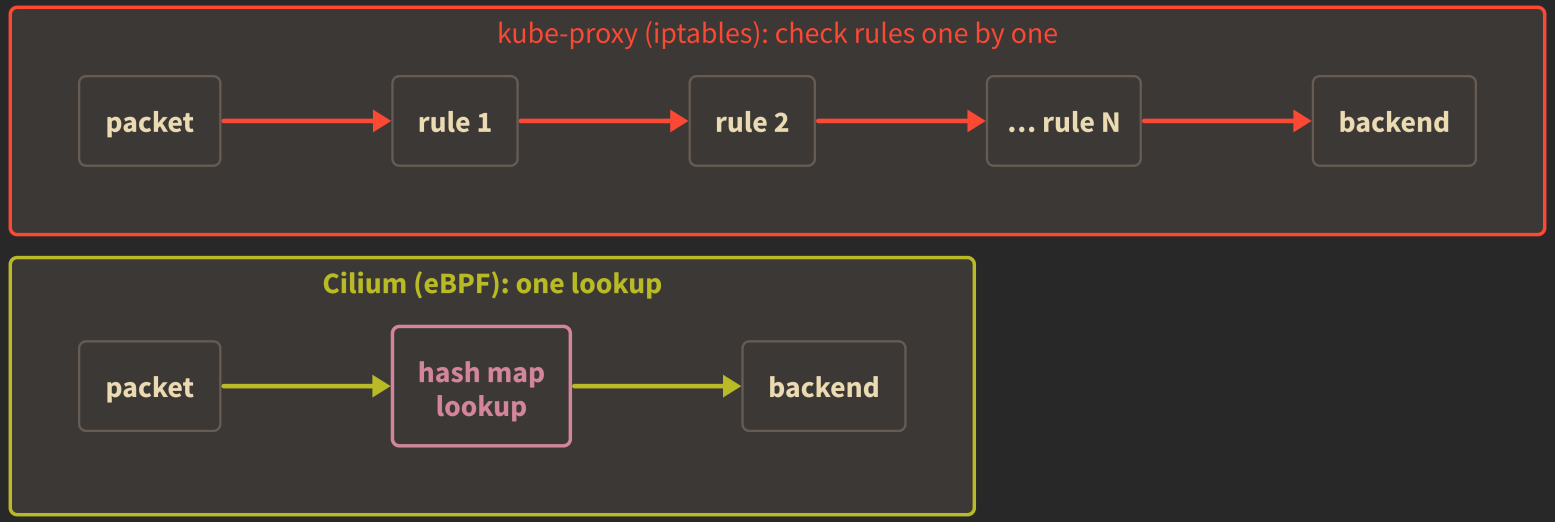
```
ping -c 2 10.0.0.2                                  # works
sudo ip -n demo link set veth1 xdp obj xdp.bpf.o sec xdp
ping -c 2 -W 1 10.0.0.2                             # silence
curl -s http://10.0.0.2:8000 | head -1              # still works
sudo ip -n demo link set veth1 xdp off
```

ICMP dies **in the driver**. TCP passes untouched. No firewall rules, no restart.

XDP vs TC vs DPDK

	XDP	TC	DPDK
Runs in	the kernel's NIC driver	the kernel, after sk_buff	**user space**
NIC needs	XDP-capable driver (most); generic mode: any NIC	any NIC	a DPDK driver; NIC **leaves** the kernel
Kernel stack & tools	kept	kept	**gone**
Directions	incoming only	both	both
CPU	only when packets arrive	only when packets arrive	cores **polling at 100%**
Typical use	DDoS drop, load balancing	policies, NAT, pods	telco: 5G UPF, VNFs

iptables vs eBPF in Kubernetes



Remember slide "Networking the old way": more services → more rules to walk. With eBPF, a lookup costs **the same for 10 services or 10,000**.

7 · eBPF in Production

The landscape



Cilium and Hubble

- A **Kubernetes network plugin (CNI)** built on eBPF
- Replaces kube-proxy: service routing through **eBPF maps**, not iptables
- Network policies based on **pod identity (labels)**, not IP addresses
- **Hubble**: see every flow, DNS query, and HTTP request between pods

Used by **GKE Dataplane V2** and many managed Kubernetes platforms. **Graduated in the CNCF** in 2023.

Tetragon and Falco: runtime security

Falco

- Watches syscalls and kernel events
- **Rules** flag suspicious behavior: "a shell started in a container"
- **Detects** and alerts
- CNCF graduated (2024)

Tetragon

- Watches process, file, and network events, **with Kubernetes context**
- **Detects and enforces** inside the kernel: can kill the process **before** the action completes
- From the Cilium team

Tetragon: block it in the kernel

```yaml
apiVersion: cilium.io/v1alpha1
kind: TracingPolicyNamespaced
metadata:
  name: block-shadow
  namespace: default
spec:
  kprobes:
  - call: "security_file_permission"
    syscall: false
    args:
    - index: 0
      type: "file"
    - index: 1
      type: "int"
    selectors:
    - matchArgs:
      - index: 0
        operator: "Equal"
        values: ["/etc/shadow"]
      matchActions:
      - action: Sigkill
```

Limits & risks

- **Strict verifier**: valid-looking code gets rejected until it can prove safety
- **Kernel versions matter**: new hooks and helpers need new kernels
- **Harder to debug** than user space: verifier logs, `bpf_printk`, `bpftool`

- **The same power for attackers**: eBPF rootkits (*ebpfkit*, DEF CON 2021), malware (*BPFDoor*, 2022)
- **Defend**: unprivileged eBPF off, `CAP_BPF` instead of full root, audit with `bpftool prog list`

8 · Getting Started

Your learning path

1. **bpftrace one-liners**: answer real questions on your own laptop today
2. **BCC tools**: execsnoop, opensnoop, biolatency, tcplife
3. **Write your own**: libbpf-bootstrap (C) or cilium/ebpf (Go)
4. **Read real projects**: Cilium, Tetragon, Falco

Everything in this talk ran on a **normal laptop**. You don't need a lab.

Live: write your own, in Go

Kernel side (C)

```c
SEC("tracepoint/sock/inet_sock_set_state")
int trace_connect(
    struct trace_event_raw_inet_sock_set_state *ctx)
{
    if (ctx->newstate != TCP_SYN_SENT)
        return 0;
    /* reserve a ring buffer slot, fill in:
       PID, process, destination IP + port */
    bpf_ringbuf_submit(e, 0);
    return 0;
}
```

User side (Go, cilium/ebpf)

```go
loadBpfObjects(&objs, nil)      // load
link.Tracepoint("sock",         // attach
    "inet_sock_set_state",
    objs.TraceConnect, nil)
rd, _ := ringbuf.NewReader(objs.Events)
rec, _ := rd.Read()   // one event
country(geo, ip)      // → country
```

bpf2go compiles the C and embeds it: **one Go binary**. Country data: IP Geolocation by DB-IP.com (CC BY 4.0).

Resources

- **ebpf.io**: the official starting point
- ***Learning eBPF***, Liz Rice (O'Reilly, 2023)
- ***BPF Performance Tools***, Brendan Gregg (2019)
- ***eBPF: Unlocking the Kernel***: the documentary (2023, YouTube)
- **bpftrace one-liner tutorial**: github.com/bpftrace/bpftrace
- **libbpf-bootstrap**: github.com/libbpf/libbpf-bootstrap

Thank you!

> "Now a hundred thousand people in the world can program the kernel thanks to BPF."
> — *eBPF: Unlocking the Kernel* (2023)

Mohammad Heidari
{t.me,github.com}/itsMohammadHeidari
itsMohammadHeidari@gmail.com

Questions?